LAB 2 REPORT

BELHIMER Mohamed Ayoub • Group 1

1. CoppeliaSim Simulator

1.3 Tasks

Task 1: CoppeliaSim Simulator Overview

According to the CoppeliaSim manual, the simulator provides:

- **Distributed Architecture:** Control robots via Lua/Python scripts, ROS/ROS 2 nodes, or Remote APIs (C++, Python).
- **Physics Engines:** Built-in support for Bullet, ODE, MuJoCo, and Vortex dynamics.
- **Sensors & Actuators:** Built-in proximity sensors, 2D/3D LiDARs, vision sensors, and encoders.
- **Modular Models:** Portable `.ttm` model files containing geometry, physics, and scripts.

Task 2: Sensor Exploration and Classification

The Model Browser under `components/sensors` contains various robotic sensors:

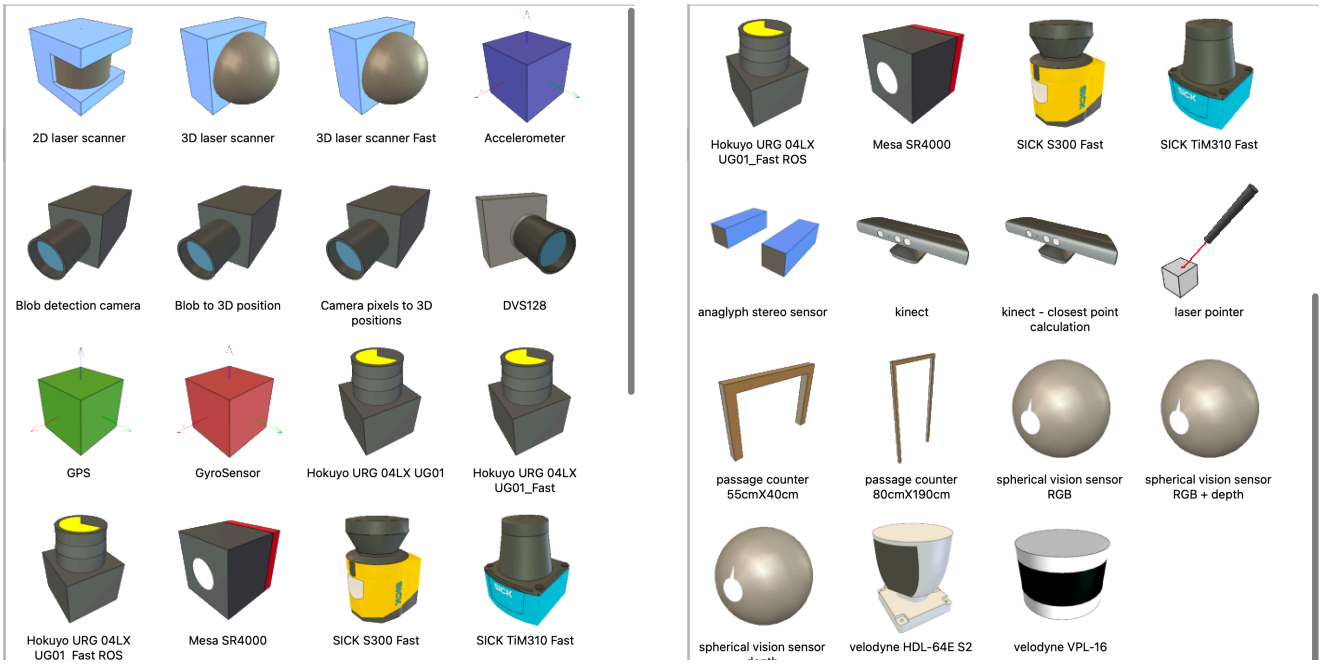


Figure 1. CoppeliaSim Model Browser displaying available sensor models under `components/sensors`.

(a) Three Identified Sensor Types:

1. **2D Laser Scanner (LiDAR)** (e.g., *Hokuyo URG 04LX UG01*)
2. **RGB-D Vision Sensor** (e.g., *Kinect*)
3. **Inertial Sensor (IMU)** (e.g., *Accelerometer / GyroSensor*)

(b) Functional Description:

- **2D Laser Scanner (LiDAR):** Measures distances to surrounding surfaces using Time-of-Flight (ToF) laser pulses across a 2D plane. It is used for obstacle avoidance, 2D SLAM, and building occupancy grid maps.
- **RGB-D Sensor (Kinect):** Combines an RGB camera with an infrared depth sensor to output color images and 3D point clouds. It is used for 3D environment mapping and visual object recognition.
- **Inertial Measurement Unit (IMU):** Measures linear acceleration with an accelerometer and angular velocity with a gyroscope. It provides high-frequency motion tracking and assists in robot state estimation.

(c) Proprioceptive vs. Exteroceptive Sensors:

- **Proprioceptive sensors:** Measure internal states of the robot (e.g., wheel encoders, motor current, battery level, accelerometers, gyroscopes).
- **Exteroceptive sensors:** Measure signals from the external environment (e.g., LiDAR, sonar, cameras, bump sensors, GPS).

Task 3: Wheeled Mobile Robot Exploration

The Model Browser under `robots/mobile` contains various mobile robotic platforms:

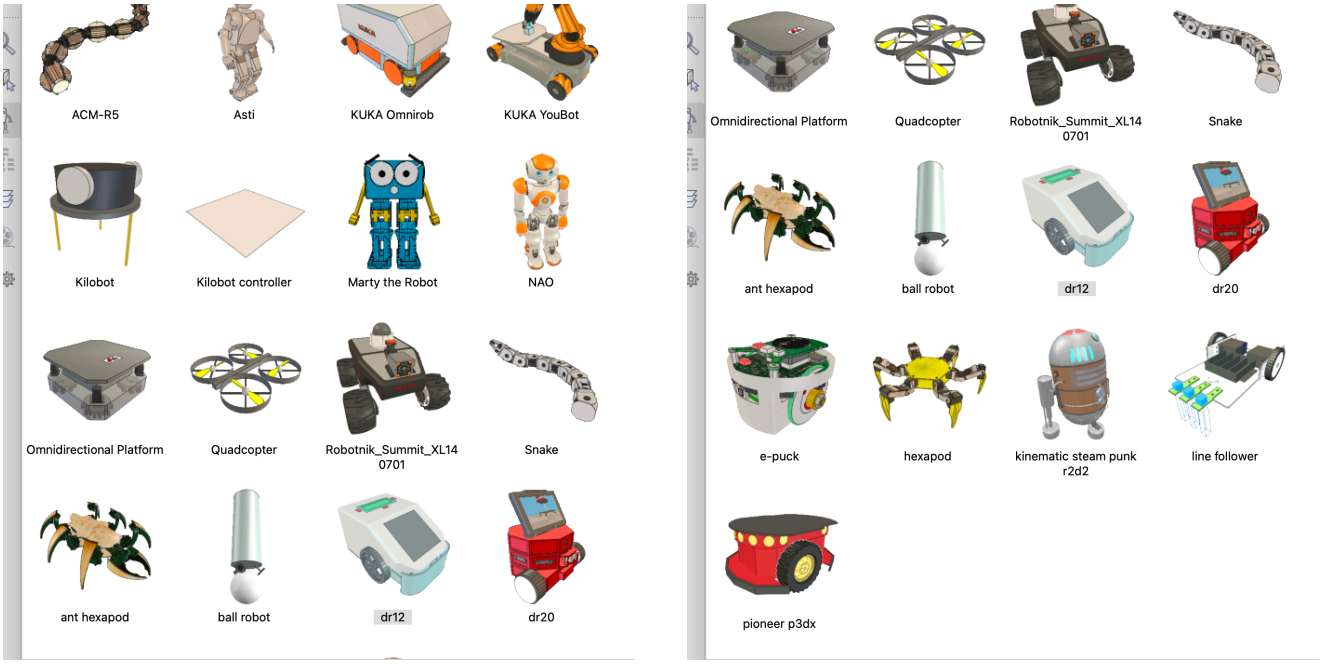


Figure 2. CoppeliaSim Model Browser displaying available mobile robots under `robots/mobile`.

(a) & (b) Identified Mobile Robot Types:

1. **Pioneer P3DX** (Differential Drive)
2. **KUKA YouBot** (Omnidirectional Drive)
3. **Robotnik Summit XL** (4WD Differential Drive)

(c) Wheel Configuration Description:

- **Pioneer P3DX (Differential Drive):** Two independently driven coaxial wheels on a central axis, with a rear passive caster for balance. Straight motion uses equal wheel velocities; turning uses a differential velocity between left and right wheels.
- **KUKA YouBot (Omnidirectional Drive):** Four Mecanum wheels (also known as Swedish wheels) with 45° passive rollers. Allows holonomic 3-DOF planar motion (instant translation in x, y and rotation θ without needing to turn its body first).
- **Robotnik Summit XL (4WD Differential Drive):** Four fixed-axis driven wheels (two per side). Turning is achieved identically to tracked tanks: the left-side and right-side wheels are driven at different speeds, forcing the tires to intentionally slip (skid) laterally across the ground to reorient.

Task 4: Robot Instantiation and Simulation Run

Robot models were dragged from the Model Browser into the scene and the simulation was started.

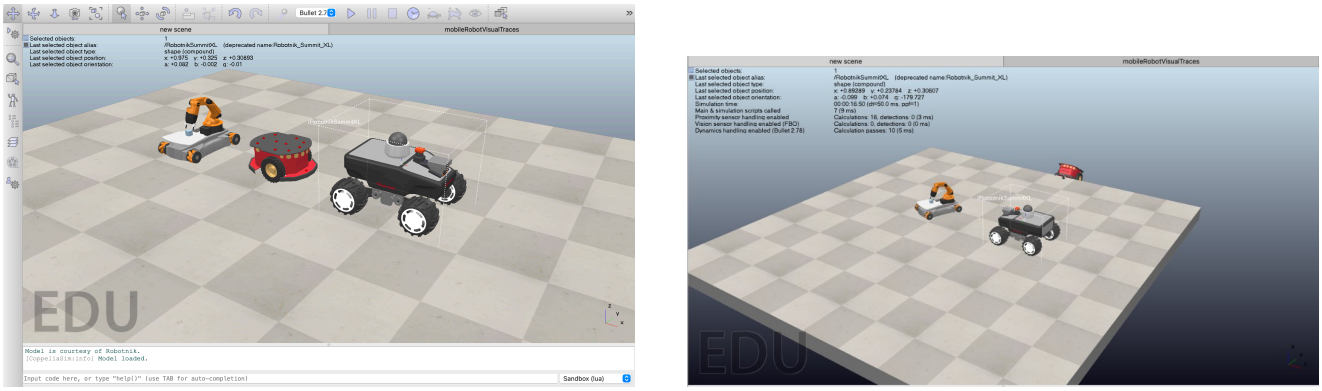


Figure 3. Robot models placed in the scene (left) and dynamic motion during simulation (right).

Task 5: Demo Scene Execution

Two pre-built demo scenes were executed:

Demo 1: `mobileRobotVisualTraces.ttt` Mobile robots navigate reactively while generating visual trajectory traces.

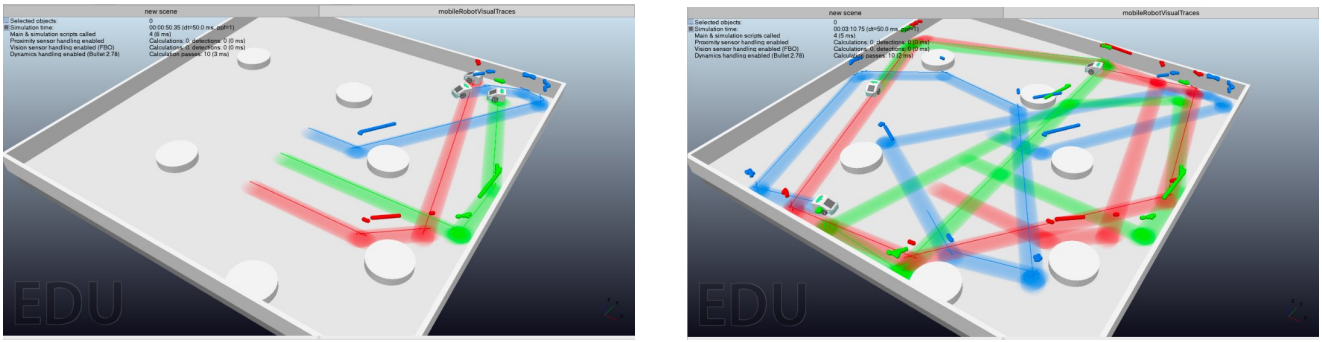


Figure 4. `mobileRobotVisualTraces` scene at $t = 50\,\text{s}$ (left) and $t = 3\,\text{min}$ (right).

Demo 2: `navigationWithinAPointCloud.ttt` A Pioneer P3DX navigates inside a 3D point cloud mesh using proximity sensor rays for obstacle detection.

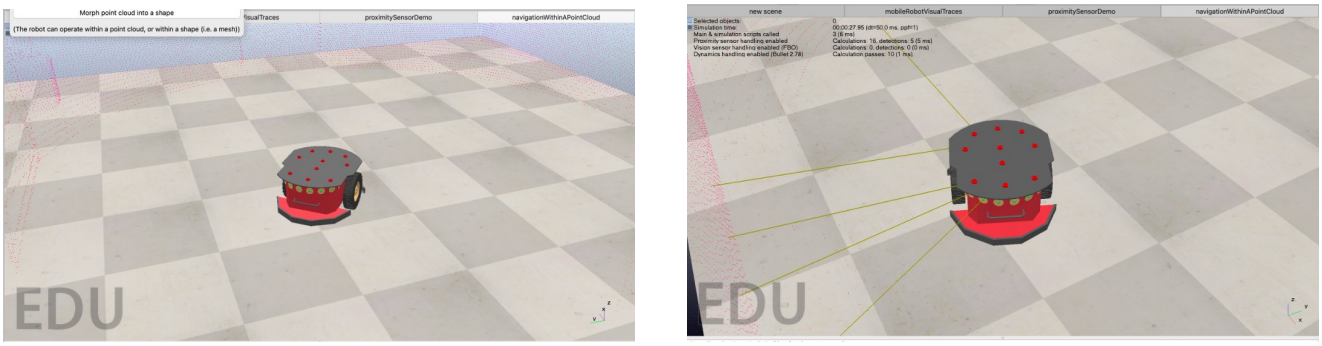


Figure 5. `navigationWithinAPointCloud` initial scene (left) and active sensor ray-casting (right).

Task 6: Simulator Tutorial Execution (BubbleRob)

The official `BubbleRob` tutorial was executed to study differential-drive robot modeling. It demonstrates two actuated wheels, a passive caster, proximity sensor ray-casting for obstacle avoidance, vision sensor processing, and real-time distance plotting.

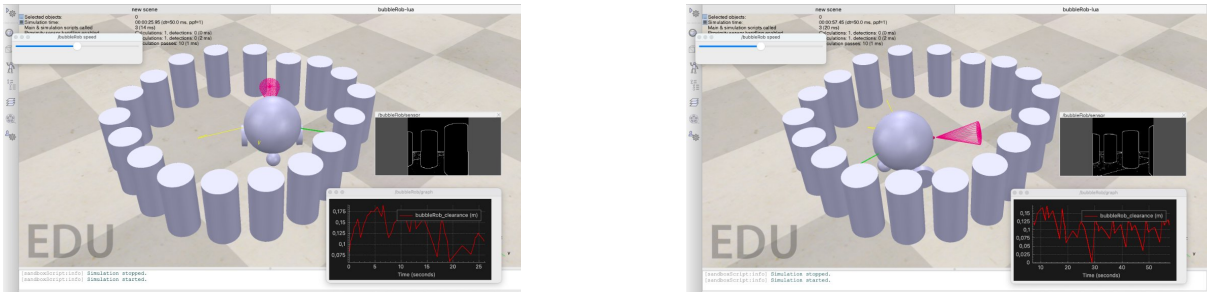


Figure 6. `BubbleRob` tutorial simulation showing proximity cone ray-casting, vision sensing, and clearance graph.

1.4 ROS 2 Interface

1.4.1 Installation and Compilation

The `sim_ros2_interface` package was configured with the lab's `interfaces.txt` and compiled using `colcon build`:

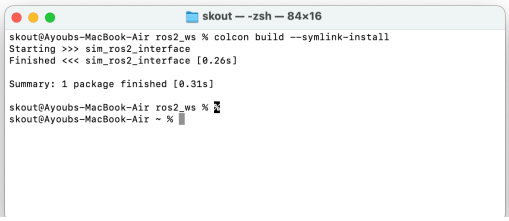


Figure 7. Successful compilation and installation of `sim_ros2_interface` via `colcon build`.

1.4.2 Testing the ROS 2 Interface

With CoppeliaSim running the `ENSIA_IMR_Lab2.ttt` scene, active nodes and simulation topics were verified:

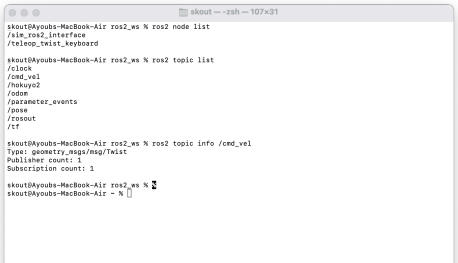


Figure 8. Active ROS 2 graph showing the `/sim_ros2_interface` node and registered simulation topics.

2. Differential Drive Robot Motion Modeling

2.2 Tasks

Task 1: Scene Initialization (`ENSIA_IMR_Lab2.ttt`)

The laboratory scene `ENSIA_IMR_Lab2.ttt` was opened in CoppeliaSim, initializing the Pioneer P3DX differential-drive mobile robot, actuated wheel joints, sensors, and the planar ground arena.

Task 2: Wheel Angular Velocity Derivation & Implementation

From the lab manual, the kinematic relationship of a differential-drive robot is defined by Equations (1) to (4):

$$v = \frac{v_R + v_L}{2} \tag{1}$$

$$\omega = \frac{v_R - v_L}{l} \tag{2}$$

$$v_R = r \cdot \dot{\phi}_R \tag{3}$$

$$v_L = r \cdot \dot{\phi}_L \tag{4}$$

where r is the wheel radius and l is the wheelbase.

Derivation of $\dot{\phi}_R$ and $\dot{\phi}_L$ from Equations (1) to (4): From Equations (1) and (2):

$$v_R + v_L = 2v$$
$$v_R - v_L = \omega \cdot l$$

Adding the two equations:

$$2v_R = 2v + \omega \cdot l \implies v_R = v + \frac{\omega \cdot l}{2}$$

Subtracting the two equations:

$$2v_L = 2v - \omega \cdot l \implies v_L = v - \frac{\omega \cdot l}{2}$$

Substituting Equations (3) and (4) ($v_R = r \cdot \dot{\phi}_R$ and $v_L = r \cdot \dot{\phi}_L$) into the expressions:

$$\dot{\phi}_R = \frac{2v + \omega \cdot l}{2r} = \frac{v}{r} + \frac{\omega \cdot l}{2r} \tag{5}$$

$$\dot{\phi}_L = \frac{2v - \omega \cdot l}{2r} = \frac{v}{r} - \frac{\omega \cdot l}{2r} \tag{6}$$

Equations (5) and (6) compute the target angular velocities for the right and left wheel joints from incoming `/cmd_vel` commands.

Implementation in Pioneer P3DX Lua Script: The derived equations were implemented in the `setVelocity_cb` callback of the Pioneer P3DX simulation script:

```lua
end

function setVelocity_cb(msg)
    -- The message provides linear velocity in m/s
    -- but coppelia receives it in rad/s, we need to convert it

    linear_cmd_vel = msg.linear.x
    angular_cmd_vel = msg. angular.z

    --  Calculate the wheel spinning speeds phi_dot_R and phi_dot_L
    --  using the provided linear and angular velocities of the robot.
    ------------------------------------------------------------------
    ------------------------------------------------------------------
    phi_dot_R = (2 * linear_cmd_vel + angular_cmd_vel * ROBOT_WIDTH) / (2 * WHEEL_RADIUS)
    phi_dot_L = (2 * linear_cmd_vel - angular_cmd_vel * ROBOT_WIDTH) / (2 * WHEEL_RADIUS)

    ------------------------------------------------------------------
    ------------------------------------------------------------------

    sim.setJointTargetVelocity(left_wheel, phi_dot_L)
    sim.setJointTargetVelocity(right_wheel, phi_dot_R)

end
```

Figure 9. Pioneer P3DX Lua script with the implemented wheel speed calculations in `setVelocity_cb`.

Task 3: Velocity Publishing via ROS 2 CLI

To verify the kinematic motion model in simulation, the scene was started in CoppeliaSim, and linear and angular velocity commands were published to the `/cmd_vel` topic at a frequency of 1 Hz using the ROS 2 command-line interface:

```
$ ros2 topic pub --rate 1 /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 1.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

Kinematic Motion Analysis: For the commanded velocities $v = 1.0\,\text{m/s}$ and $\omega = 1.8\,\text{rad/s}$, with Pioneer P3DX parameters $r = 0.0975\,\text{m}$ (wheel radius) and $l = 0.33\,\text{m}$ (wheelbase):

$$\dot{\phi}_R = \frac{1.0}{0.0975} + \frac{1.8 \cdot 0.33}{2 \cdot 0.0975} \approx 10.256 + 3.046 = 13.302\,\text{rad/s}$$

$$\dot{\phi}_L = \frac{1.0}{0.0975} - \frac{1.8 \cdot 0.33}{2 \cdot 0.0975} \approx 10.256 - 3.046 = 7.210\,\text{rad/s}$$

Because $\dot{\phi}_R > \dot{\phi}_L$, the right wheel spins faster than the left wheel, resulting in a counter-clockwise circular trajectory with turning radius $R = v/\omega = 1.0/1.8 \approx 0.556\,\text{m}$.



Figure 10. Publishing continuous Twist velocity commands ($v = 1.0\,\text{m/s}, \omega = 1.8\,\text{rad/s}$) to `/cmd_vel` at 1 Hz.

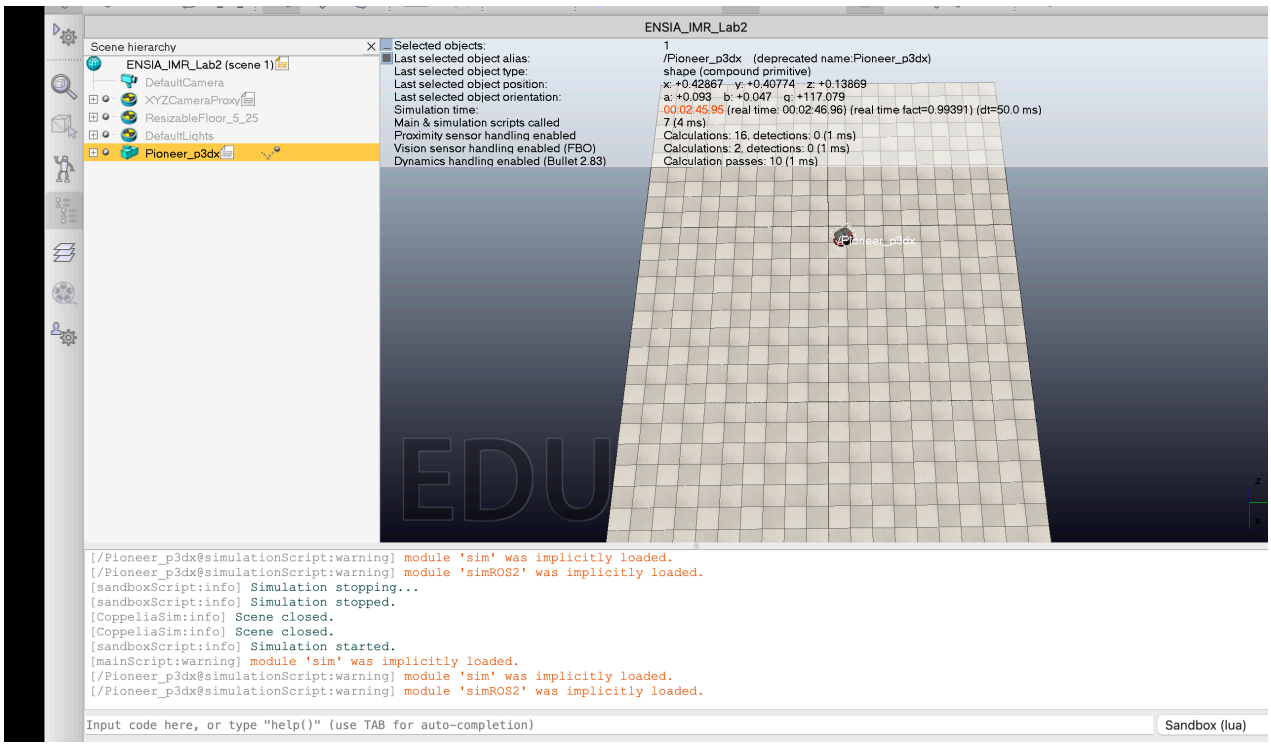


Figure 11. Pioneer P3DX executing the circular trajectory in CoppeliaSim corresponding to the received Twist velocities.

Task 4: Keyboard Teleoperation Verification

The `teleop_twist_keyboard` node was executed in the terminal to interactively drive and steer the Pioneer P3DX robot in real time:

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Keystrokes were captured by the node and published directly to `/cmd_vel`, validating manual teleoperation and dynamic motion in the simulated environment.

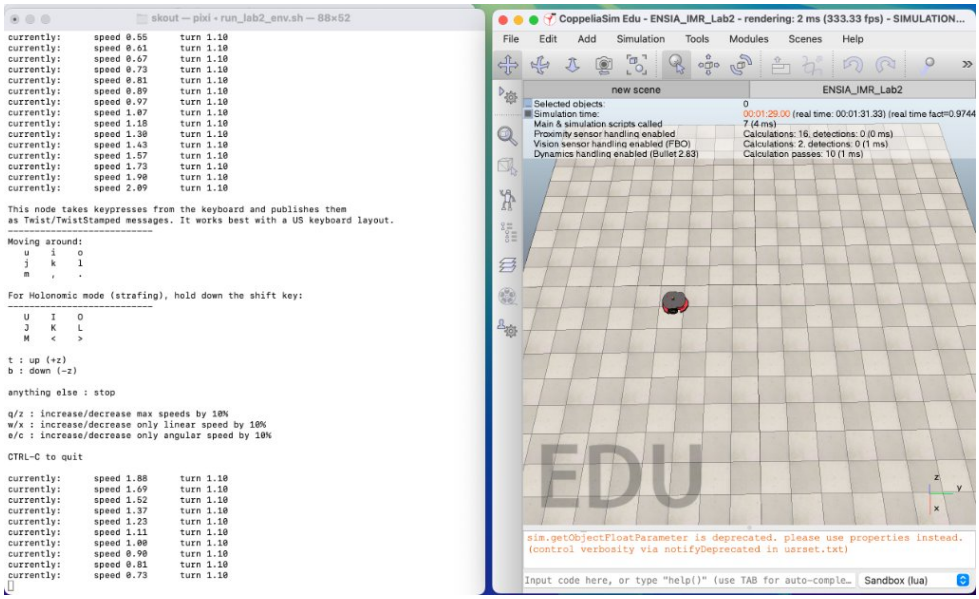


Figure 12. Initial teleoperation setup: terminal running `teleop_twist_keyboard` side-by-side with CoppeliaSim ($t = 01:29$).

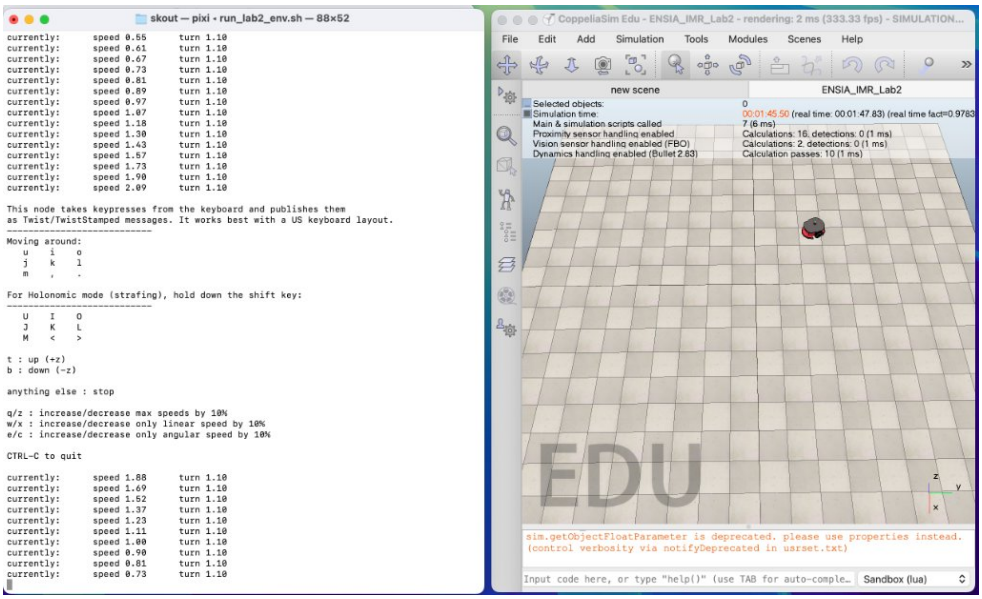


Figure 13. Manual trajectory execution: Pioneer P3DX driven forward and steered across the arena floor ($t = 01:45$).

Task 5: Odometry Kinematic Model Implementation in Lua

The discrete-time forward kinematic model (Equation 5 from the lab manual) was implemented in the Pioneer P3DX Lua script under `getRobotOdometry()` to compute dead-reckoning pose estimates from wheel encoder speeds:

$$x_{t+\Delta t} = x_t + v\cos\theta_t \cdot \Delta t \tag{7}$$
$$y_{t+\Delta t} = y_t + v\sin\theta_t \cdot \Delta t \tag{8}$$
$$\theta_{t+\Delta t} = \theta_t + \omega \cdot \Delta t \tag{9}$$

where $v = \frac{v_R+v_L}{2}$, $\omega = \frac{v_R-v_L}{l}$, and $\Delta t =$ `sim.getSimulationTimeStep()`.

```lua
--  Calculate the robot pose using the kinematik model
------------------------------------------------------------------------
------------------------------------------------------------------------

-- Robot Velocity (Twist) in the world frame

local vx_world = linear_velocity * math.cos(theta)
local vy_world = linear_velocity * math.sin(theta)
local yaw_world = angular_velocity

-- Robot Pose [x y theta] in the world frame

robot_odom_pose[1] = robot_odom_pose[1] + vx_world * dt
robot_odom_pose[2] = robot_odom_pose[2] + vy_world * dt
robot_odom_pose[3] = robot_odom_pose[3] + yaw_world * dt



------------------------------------------------------------------------
------------------------------------------------------------------------
```

Figure 14. Implemented odometric pose integration in the Pioneer P3DX Lua script (`getRobotOdometry`).

Task 6: Odometry vs. Ground Truth Pose Verification via Topic Echo

To verify the implemented odometry model against ground truth, the simulation was started, the Pioneer was teleoperated using `teleop_twist_keyboard`, and both topics were echoed simultaneously in separate terminals:

```
$ ros2 topic echo /odom    # Calculated odometric pose (with encoder noise)
$ ros2 topic echo /pose    # Ground truth pose from CoppeliaSim
```

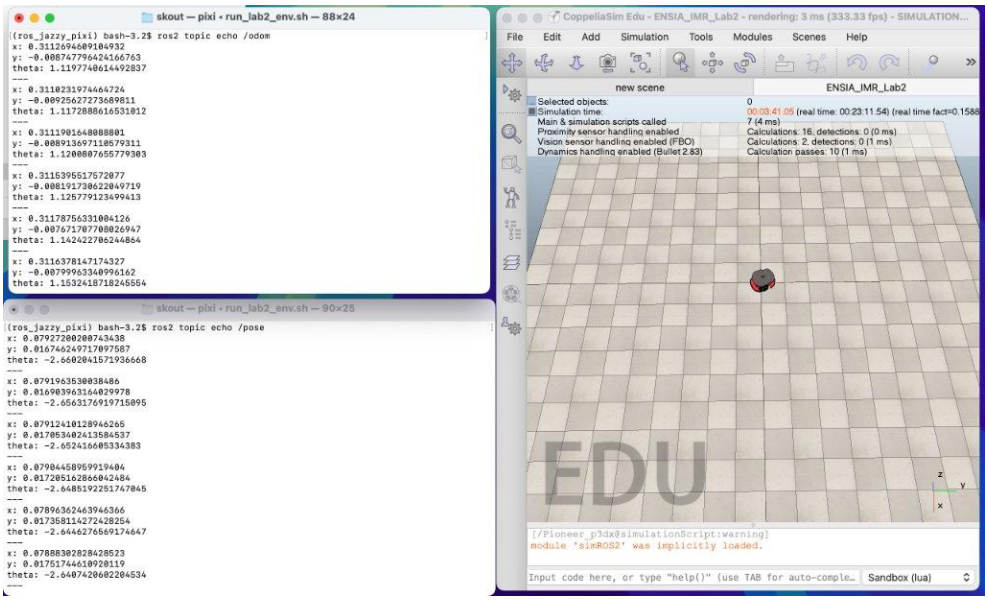


Figure 15. Simultaneous topic echo: calculated noisy odometry (`/odom`, top) versus ground truth (`/pose`, bottom) during robot motion in CoppeliaSim.

Task 7: ROS 2 Bag Recording, Inspection, and Playback

All active ROS 2 topics published during the robot simulation and teleoperation were recorded into a standard ROS 2 bag file using `ros2 bag record`:

```
$ cd ~/ENSIA/Y4/S7/ROBOTICS/LABS/2
$ ros2 bag record -a -o Lab2_BELHIMER_Mohamed_Ayoub
```

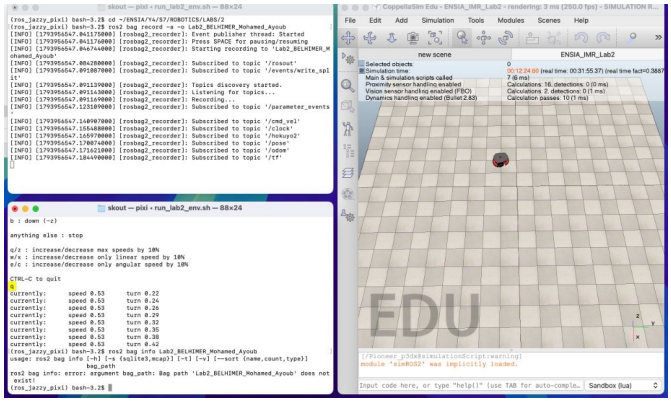


Figure 16. Simultaneous topic recording using `ros2 bag record -a` while teleoperating the Pioneer P3DX in CoppeliaSim.

The recorded bag structure and topic metadata were verified via `ros2 bag info`:

```
$ ros2 bag info Lab2_BELHIMER_Mohamed_Ayoub

Files:             Lab2_BELHIMER_Mohamed_Ayoub_0.mcap
Bag size:          4.9 MiB
Storage id:        mcap  |  ROS Distro: jazzy
Duration:          80.389s  |  Messages: 7773
Topic information:
  /clock       | Count: 1540 | Type: rosgraph_msgs/msg/Clock
  /cmd_vel     | Count: 56   | Type: geometry_msgs/msg/Twist
  /hokuyo2     | Count: 1541 | Type: sensor_msgs/msg/LaserScan
  /odom        | Count: 1540 | Type: geometry_msgs/msg/Pose2D
  /pose        | Count: 1540 | Type: geometry_msgs/msg/Pose2D
  /tf          | Count: 1541 | Type: tf2_msgs/msg/TFMessage
  /rosout      | Count: 15   | Type: rcl_interfaces/msg/Log
```

To replay the recorded mission for offline evaluation, topic monitoring, or visualization, the bag is replayed using `ros2 bag play`:

```
$ cd ~/ENSIA/Y4/S7/ROBOTICS/LABS/2
$ ros2 bag play Lab2_BELHIMER_Mohamed_Ayoub --loop
```

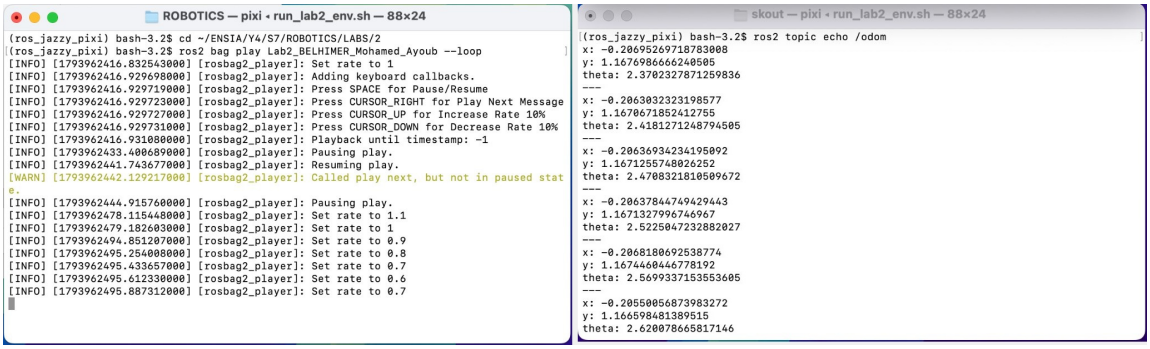


Figure 17. Offline mission playback verification: interactive playback execution with rate adjustment (left) alongside live topic subscription via `ros2 topic echo /odom` (right).

Task 8: PlotJuggler Trajectory and Pose Visualization

PlotJuggler was launched via ROS 2 to visualize and analyze the temporal and spatial evolution of the robot state recorded in `Lab2_BELHIMER_Mohamed_Ayoub`:

```
$ ros2 run plotjuggler plotjuggler
```

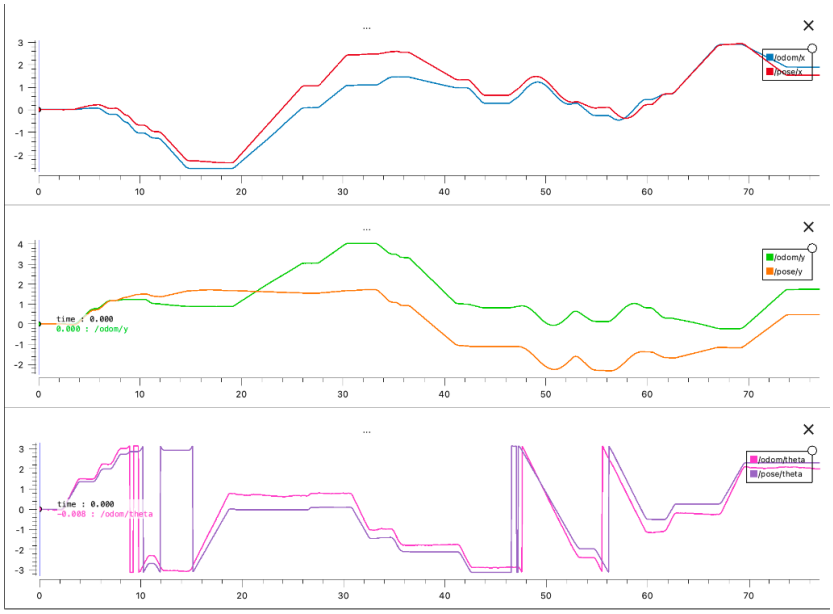


Figure 18. PlotJuggler temporal tracking: comparing calculated odometry (`/odom`) against ground truth (`/pose`) for $x(t)$ (top), $y(t)$ (middle), and $\theta(t)$ (bottom) over the 80-second mission.

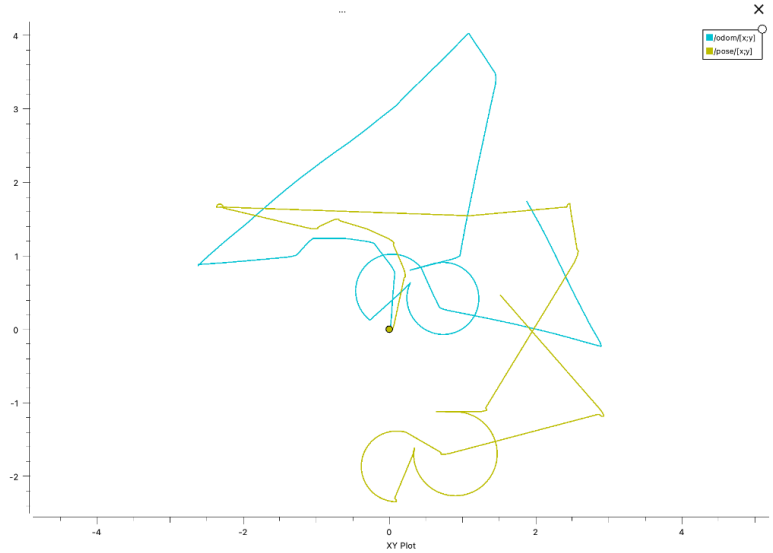


Figure 19. PlotJuggler 2D Cartesian path (y vs. x): calculated odometric path (`/odom`, cyan) versus actual ground truth path (`/pose`, yellow-green) starting at $(0, 0)$.

Task 9: Final Pose Error Computation

To quantify the accumulated dead-reckoning drift over the recorded teleoperation trajectory, the final pose from the odometry calculation was evaluated against the true simulator pose using the error metrics specified in the lab manual:

$$\text{Position_Error} = \sqrt{(x_{\text{true}} - x_{\text{odom}})^2 + (y_{\text{true}} - y_{\text{odom}})^2} \tag{10}$$

$$\text{Orientation_Error} = \sqrt{(\theta_{\text{true}} - \theta_{\text{odom}})^2} \tag{11}$$

Extracting the terminal state at $t = 80.39\,\text{s}$ yields:

- **Ground Truth Final Pose (`/pose`):**

$$x_{\text{true}} = 1.5209\,\text{m}, \quad y_{\text{true}} = 0.4623\,\text{m}, \quad \theta_{\text{true}} = 2.2948\,\text{rad}\ (131.48^\circ)$$

- **Calculated Odometry Final Pose (`/odom`):**

$$x_{\text{odom}} = 1.8814\,\text{m}, \quad y_{\text{odom}} = 1.7406\,\text{m}, \quad \theta_{\text{odom}} = 1.9979\,\text{rad}\ (114.47^\circ)$$

Substituting these coordinates into Equations (10) and (11):

$$\begin{aligned}\text{Position_Error} &= \sqrt{(1.5209 - 1.8814)^2 + (0.4623 - 1.7406)^2} \\ &= \sqrt{(-0.3605)^2 + (-1.2783)^2} = \sqrt{0.1300 + 1.6340} = \mathbf{1.3281}\,\text{m}\end{aligned}$$

$$\text{Orientation_Error} = \sqrt{(2.2948 - 1.9979)^2} = |0.2969| = \mathbf{0.2969}\,\text{rad}\ (\mathbf{17.01}^\circ)$$

Task 10: Results Analysis and Conclusion

Analysis of Results: At the beginning of the simulation ($t = 0$), the calculated odometry (`/odom`) matches the ground truth pose (`/pose`) at the origin $(0, 0, 0)$. As the robot moves, the odometric trajectory follows the general shape of the robot motion, including the turns and circular loops.

However, as time progresses, the calculated odometry gradually drifts away from the true robot pose, reaching a final position error of 1.33 m and an orientation error of 17.01° after 80 seconds. This drift occurs because:

- Gaussian noise added to the wheel encoder readings accumulates with every integration step ($\Delta t = 50\,\text{ms}$).
- Any error in the estimated heading θ misaligns the calculated velocity components ($v\cos\theta, v\sin\theta$), causing position error to grow continuously.

Conclusion: This experiment demonstrates that open-loop wheel odometry is only reliable for short periods. Because odometry errors accumulate over time without bound, mobile robots cannot rely solely on wheel odometry for localization and require external sensors (such as LiDAR, cameras, or GPS) to periodically correct the accumulated drift.